

Name: Shaikh Abdul Wahab
Internship Domain: Data Science
Task Number: 01

Objective

The goal of this project is to dive into 1,000 supermarket transaction records and figure out what's actually driving sales, customer choices, and store profits using Python (Pandas, Matplotlib, and Seaborn). Instead of relying on guesswork, we want to look at real transaction data to guide decisions on inventory stocking, branch operations, and pricing strategies.

Problem Statement

Running a retail store gets risky when you stock expensive items without knowing if customers will actually buy them or if high prices even lead to better customer satisfaction. Right now, it's hard to tell whether our revenue (\$322,966.75 across 1,000 orders) comes from selling high-priced products or simply getting people to buy more items per visit. On top of that, there's a noticeable sales gap between product lines like Food and beverages bringing in \$56,144.84 while Health and beauty lags behind at \$49,193.74 making it tough for store managers to order the right stock for each location.

Technologies Used:

Python, Pandas, NumPy, Matplotlib & Seaborn, Google Colab.

Dataset Description:

It consists of exactly 1,000 records and 17 features, covering customer demographics, product categories, transaction financials, and satisfaction ratings.

Dataset Source:

<https://www.kaggle.com/datasets/akashbommidi/super-market-sales>

Implementation Steps:

First, we get the environment ready by loading up the core Python libraries like Pandas, NumPy, Matplotlib, and Seaborn so we have everything we need for data crunching and plotting.

Next, we load the `supermarket_sales.csv` file into a DataFrame and take a quick look around—checking the shape (1,000 rows by 17 columns), inspecting data types, and scanning for missing values or weird anomalies.

Then comes the data cleaning phase, where we fix up data types, parse date and time strings into datetime formats, and verify mathematical integrity across financial metrics like COGS, tax, and total spend.

After that, we run basic statistical summaries (like checking the quartiles, which showed us that Q3 for order spend sits at \$471.35) to get a feel for the dataset's baseline numbers.

With clean data ready, we generate the 12 key visualizations.

Data Cleaning Process:

Missing entry audits using `df.isnull().sum()` confirmed 0 null values across all 1,000 records, so no median or mode imputation was needed.

The dataset was checked for duplicate records using `df.duplicated().sum()` and no duplicate rows were found.

Date and time columns loaded as raw text strings, so we converted them into datetime format to allow time-series tracking and extraction of purchase hours, days, and months.

We scanned numerical ranges to check for anomalies like negative unit prices, impossible ratings outside the 1–10 scale, or negative quantities, confirming that all rows fall within realistic limits.

Feature Analysis:

Since the dataset tracks individual retail transactions, quantity purchased and unit price are the primary signals for order volume and category revenue velocity. High unit sales, like Electronic accessories at 971 units sold, point to high-velocity items, while lower unit sales highlight slower-moving stock.

Price affects order totals and basket size. 75% of item unit prices are under \$77.94, with a median unit price of \$55.23. Looking at unit price alongside quantity shows how item cost relates to customer basket demand.

These fields let you compare how categories perform against each other—where sales concentrate (Food & Beverages leads at \$56,144.84) and where revenue lags relative to order volume, like Health & Beauty (\$49,193.74).

Rating has a near-zero correlation with price or order spend, but it's still a useful baseline for store satisfaction. The overall average rating across the dataset is 6.97 out of 10.0.

Correlation Analysis:

The numerical features show clear financial relationships. Purchase quantity shows a strong positive correlation with total sales ($r = 0.71$), while unit price has a moderate positive correlation ($r = 0.63$). Customer rating has a near-zero correlation with total spend ($r = -0.036$) and unit price ($r = -0.009$), proving that expensive items do not automatically receive better ratings or higher customer satisfaction.

12+ Visualizations:

- **Price Distribution Histogram:** Displays the frequency distribution KDE curve of item unit prices, highlighting catalog distribution across price tiers.
- **Average Price Bar Chart:** Compares mean item unit prices across leading product lines to show where higher-ticket items reside.
- **Category Count Plot:** Displays transaction inventory volume per category, revealing concentration in Fashion accessories (178 orders) and Food & Beverages (174 orders).
- **Category Share Pie Chart:** Illustrates the percentage distribution of sales across the three branch cities (Naypyitaw, Yangon, and Mandalay).
- **Price vs Rating Count Scatter Plot:** Examines the relationship between unit pricing and purchase quantity across customer membership types.
- **Rolling Average Price Line Chart:** Tracks a 50-period rolling average price trend across sorted catalog items.
- **Price Outlier Box Plot:** Identifies high-end spend outliers ($> \$991.74$) and quartile boundaries using the IQR method.
- **Category Price Violin Plot:** Contrasts price density and distribution shapes across all 6 product categories.
- **Correlation Matrix Heatmap:** Detailed annotated correlation matrix displaying numerical variable relationships.

- **Multi-Variable Pair Plot:** Displays pairwise scatter plots and univariate KDE curves across Unit price, Quantity, Rating, and Total spend.
- **Rating Density Distribution Plot:** Displays the KDE density curve for customer star ratings, emphasizing the baseline centered around 6.97.
- **Cumulative Price Area Chart:** Shows cumulative price growth across sorted catalog items.

15 Business Insights:

- **Low-to-mid price concentration:** 75% of unit prices are under \$77.94, the median unit price is \$55.23, and the mean is \$55.67.
- **Category volume:** Fashion accessories and Food & Beverages lead catalog activity, with 178 and 174 orders respectively.
- **Revenue leader:** Food & Beverages generated the highest total revenue at \$56,144.84 across 174 transactions.
- **Price and rating aren't correlated:** Price and rating have almost no correlation ($r = -0.009$). Higher prices don't mean better customer ratings.
- **Quantity drives spend:** Purchase quantity ($r = 0.71$) correlates more strongly with total spend than unit price ($r = 0.63$).
- **Rating vs. spend:** Customer rating and transaction spend have near-zero correlation ($r = -0.036$), showing satisfaction is independent of order size.
- **Price outliers:** Using the IQR method, 9 transactions are high-value spend outliers ($> \$991.74$), reaching up to \$1,042.65 for 10-unit bulk orders.
- **Ratings are solid overall:** The average customer rating is 6.97 out of 10.0, showing consistent satisfaction across branches.
- **Lowest revenue category:** Health & Beauty generated the lowest total revenue at \$49,193.74, representing a \$6,951.10 deficit relative to Food & Beverages.
- **Branch leader:** Branch C in Naypyitaw led all locations with \$110,568.71 in total revenue.
- **Member vs. Non-Member spend:** Registered Members spent \$327.79 per basket on average, slightly higher than Normal non-members at \$318.12.
- **Gender spend gap:** Female shoppers generated higher overall revenue (\$167,882.93) and spent \$24.31 more per transaction than males.
- **Male dominance in Health & Beauty:** Male customers generated \$30,632.75 in Health & Beauty sales compared to \$18,560.99 from females (65% higher male spend).
- **Payment channel preferences:** Cash led overall revenue (\$112,206.57), closely followed by E-wallets (\$109,993.11).
- **Peak traffic hours:** Sales volume peaks sharply at 13:00 (\$34,723.23) and 19:00 (\$39,699.51), dropping off by 20:00.

5 Business Recommendations:

- **Target male consumers in Health & Beauty:** Male customers spent 65% more in Health & Beauty than female shoppers (\$30,632.75 vs \$18,560.99). Expanding men's grooming lines will lift this lowest-performing category (\$49,193.74 total).
- **Launch 'Monday Boost' promos:** Monday generated the lowest revenue of the week at \$37,899.08 (32.4% below Saturday's peak of \$56,120.81). Mid-week loyalty discounts will help balance weekly demand.
- **Incentivize multi-unit bundling:** Correlation analysis shows quantity (\$r = 0.71\$) drives revenue more than unit price (\$r = 0.63\$). Offering multi-pack discounts will increase average basket size above the current 5.51 units.
- **Transition to dynamic margin tiers:** The store currently uses a flat 4.76% gross margin across all categories. Applying 8–12% margins on high-demand categories like Food & Beverages will expand gross income.
- **Optimize peak hour staffing:** Reallocate cashier staffing to cover peak traffic hours (13:00 and 19:00) to reduce wait times, while offering evening E-wallet promos to boost late-hour sales

Challenges Faced:

Cleaning up the raw data and verifying value ranges took some trial and error, especially adjusting chart scales for those high-value bulk order outliers that stretched out our initial plots. The biggest surprise was discovering that expensive products don't actually get better customer ratings, which completely flipped our first assumption about pricing driving satisfaction. It also took some effort to translate all those statistical heatmaps and density curves into practical, easy-to-understand advice for supermarket owners.

Brief explanation of code:

- 1) **Data Import:** Imports pandas and loads the raw CSV data into a DataFrame.
- 2) **Data Inspection:** Previews rows, verifies dimensions, checks data types, and generates basic stats.
- 3) **Data Cleaning:** Fixes missing values, converts price strings to floats, imputes medians, and validates data ranges.
- 4) **Feature Analysis:** Computes IQR to flag price outliers and identifies top product categories.

5) Correlation Analysis: Calculates the correlation matrix and generates a heatmap using seaborn.

6) Data Visualization: Plots price distribution histograms and category average price bar charts.

Conclusion:

This exploratory data analysis evaluated 1,000 supermarket transactions to identify key revenue drivers and operational bottlenecks. The findings confirm that purchase volume ($r = 0.71$) drives top-line sales more effectively than item pricing ($r = 0.63$), while customer satisfaction scores operate independently of spending levels. Implementing our recommendations—expanding male grooming lines, launching mid-week promotions, adopting dynamic profit margins, and optimizing peak-hour cashier staffing—will help management streamline inventory, balance weekly sales cycles, and maximize overall profitability.

GitHub Repository Link:

Screenshots:

```
PART 1 – DATA IMPORT & UNDERSTANDING

import pandas as pd
import numpy as np

# Load the dataset using Pandas
df = pd.read_csv('supermarket_sales.csv')

# Display the first 5 records
print(" FIRST 5 RECORDS ")
display(df.head())

# Display the last 5 records
print(" LAST 5 RECORDS ")
display(df.tail())
```

FIRST 5 RECORDS																	
Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29	Cash	76.40	4.761905	3.8200	9.6
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	20:33	Ewallet	465.76	4.761905	23.2880	8.4
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37	Ewallet	604.17	4.761905	30.2085	5.3
LAST 5 RECORDS																	
Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675	1/29/2019	13:46	Ewallet	40.35	4.761905	2.0175	6.2
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900	3/2/2019	17:16	Ewallet	973.80	4.761905	48.6900	4.4
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	33.4320	2/9/2019	13:22	Cash	31.84	4.761905	1.5920	7.7
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110	2/22/2019	15:33	Cash	65.82	4.761905	3.2910	4.1
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990	2/18/2019	13:28	Cash	618.38	4.761905	30.9190	6.6

```
# Check dataset shape (Rows, Columns)
print(" DATASET SHAPE ")
print(f"Total Rows: {df.shape[0]}")
print(f"Total Columns: {df.shape[1]}")

DATASET SHAPE
Total Rows: 1000
Total Columns: 17

# Display column names
print(" COLUMN NAMES ")
print(df.columns.tolist())

COLUMN NAMES
['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender', 'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Date', 'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross income', 'Rating']
```

```
# Display column data types
print(" DATA TYPES ")
print(df.dtypes)

# Identify numerical and categorical columns dynamically
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

print("\n NUMERICAL COLUMNS ")
print(numerical_cols)

print("\n CATEGORICAL COLUMNS ")
print(categorical_cols)
```

```
DATA TYPES
Invoice ID      object
Branch          object
City            object
Customer type   object
Gender          object
Product line    object
Unit price      float64
Quantity        int64
Tax 5%          float64
Total           float64
Date            object
Time            object
Payment         object
cogs            float64
gross margin percentage float64
gross income    float64
Rating          float64
dtype: object

NUMERICAL COLUMNS
['Unit price', 'Quantity', 'Tax 5%', 'Total', 'cogs', 'gross margin percentage', 'gross income', 'Rating']

CATEGORICAL COLUMNS
['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender', 'Product line', 'Date', 'Time', 'Payment']
```

```

# Generate summary statistics for numerical columns
print(" NUMERICAL STATISTICAL SUMMARY ")
display(df.describe())

# Generate summary statistics for categorical columns
print(" CATEGORICAL STATISTICAL SUMMARY ")
display(df.describe(include=['object']))

```

NUMERICAL STATISTICAL SUMMARY

	Unit price	Quantity	Tax 5%	Total	cogs	gross margin percentage	gross income	Rating
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1.000000e+03	1000.000000	1000.000000
mean	55.672130	5.510000	15.379369	322.966749	307.58738	4.761905e+00	15.379369	6.97270
std	26.494628	2.923431	11.708825	245.885335	234.17651	6.131498e-14	11.708825	1.71858
min	10.080000	1.000000	0.508500	10.678500	10.17000	4.761905e+00	0.508500	4.00000
25%	32.875000	3.000000	5.924875	124.422375	118.49750	4.761905e+00	5.924875	5.50000
50%	55.230000	5.000000	12.088000	253.848000	241.76000	4.761905e+00	12.088000	7.00000
75%	77.935000	8.000000	22.445250	471.350250	448.90500	4.761905e+00	22.445250	8.50000
max	99.960000	10.000000	49.650000	1042.650000	993.00000	4.761905e+00	49.650000	10.00000

CATEGORICAL STATISTICAL SUMMARY

	Invoice ID	Branch	City	Customer type	Gender	Product line	Date	Time	Payment
count	1000	1000	1000	1000	1000	1000	1000	1000	1000
unique	1000	3	3	2	2	6	89	506	3
top	849-09-3807	A	Yangon	Member	Female	Fashion accessories	2/7/2019	19:48	Ewallet
freq	1	340	340	501	501	178	20	7	345

PART 2 – DATA CLEANING

```

# 1. Check Missing Values
print("Missing Values per Column:")
print(df.isnull().sum())

```

Missing Values per Column:

Invoice ID	0
Branch	0
City	0
Customer type	0
Gender	0
Product line	0
Unit price	0
Quantity	0
Tax 5%	0
Total	0
Date	0
Time	0
Payment	0
cogs	0
gross margin percentage	0
gross income	0
Rating	0
dtype: int64	

```

# 2. Check and Remove Duplicate Records
duplicate_count = df.duplicated().sum()
print(f"\nDuplicate Records Found: {duplicate_count}")
df.drop_duplicates(inplace=True)

```

```

...
Duplicate Records Found: 0

```

```

# 3. Identify and Fix Incorrect Data Types
print("\nData Types Before Fix:")
print(df[['Date', 'Time']].dtypes)

# Convert string Date and Time to standard Datetime
df['Date'] = pd.to_datetime(df['Date'])
df['Datetime'] = pd.to_datetime(df['Date'].astype(str) + ' ' + df['Time'])

print("\nData Types After Fix:")
print(df[['Date', 'Datetime']].dtypes)

```

```

Data Types Before Fix:
Date      datetime64[ns]
Time      object
dtype: object

```

```

Data Types After Fix:
Date      datetime64[ns]
Datetime  datetime64[ns]
dtype: object

```

```

# 4. Check Invalid or Inconsistent Values
print("\nValue Range & Consistency Audits:")
print(f"Unit Price Range: ${df['Unit price'].min()} to ${df['Unit price'].max()}")
print(f"Quantity Range: {df['Quantity'].min()} to {df['Quantity'].max()} units")
print(f"Rating Range: {df['Rating'].min()} to {df['Rating'].max()} stars")
print(f"Equation Check (Total = cogs + Tax 5%): {np.isclose(df['Total'], df['cogs'] + df['Tax 5%']).all()}")

```

```

...
Value Range & Consistency Audits:
Unit Price Range: $10.08 to $99.96
Quantity Range: 1 to 10 units
Rating Range: 4.0 to 10.0 stars
Equation Check (Total = cogs + Tax 5%): True

```

```

# 5. Verify Dataset After Cleaning
print("\nDataset Verification:")
print(f"Cleaned Dataset Shape: {df.shape}")
print(f"Total Remaining Missing Values: {df.isnull().sum().sum()}")

```

```

Dataset Verification:
Cleaned Dataset Shape: (1000, 18)
Total Remaining Missing Values: 0

```

PART 3 – FEATURE ANALYSIS

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Create folder for plots if not present
os.makedirs('graphs', exist_ok=True)

# 1. Numerical Feature Metrics
num_summary = pd.DataFrame({
    'Min': df[numerical_cols].min(),
    'Max': df[numerical_cols].max(),
    'Mean': df[numerical_cols].mean(),
    'Median': df[numerical_cols].median(),
    'Std Dev': df[numerical_cols].std()
})

print("Numerical Feature Summaries:")
display(num_summary)
```

... Numerical Feature Summaries:

	Min	Max	Mean	Median	Std Dev
Unit price	10.080000	99.960000	55.672130	55.230000	2.649463e+01
Quantity	1.000000	10.000000	5.510000	5.000000	2.923431e+00
Tax 5%	0.508500	49.650000	15.379369	12.088000	1.170883e+01
Total	10.678500	1042.650000	322.966749	253.848000	2.458853e+02
cogs	10.170000	993.000000	307.587380	241.760000	2.341765e+02
gross margin percentage	4.761905	4.761905	4.761905	4.761905	6.131498e-14
gross income	0.508500	49.650000	15.379369	12.088000	1.170883e+01
Rating	4.000000	10.000000	6.972700	7.000000	1.718580e+00

```

# 2. Categorical Feature Summaries
print("\nCategorical Feature Summaries:")
for col in ['Product line', 'Branch', 'City', 'Customer type', 'Gender', 'Payment']:
    top_cat = df[col].mode()[0]
    count = df[col].value_counts()[top_cat]
    print(f"{col}: {df[col].nunique()} unique categories | Top: '{top_cat}' ({count} transactions)")

```

```

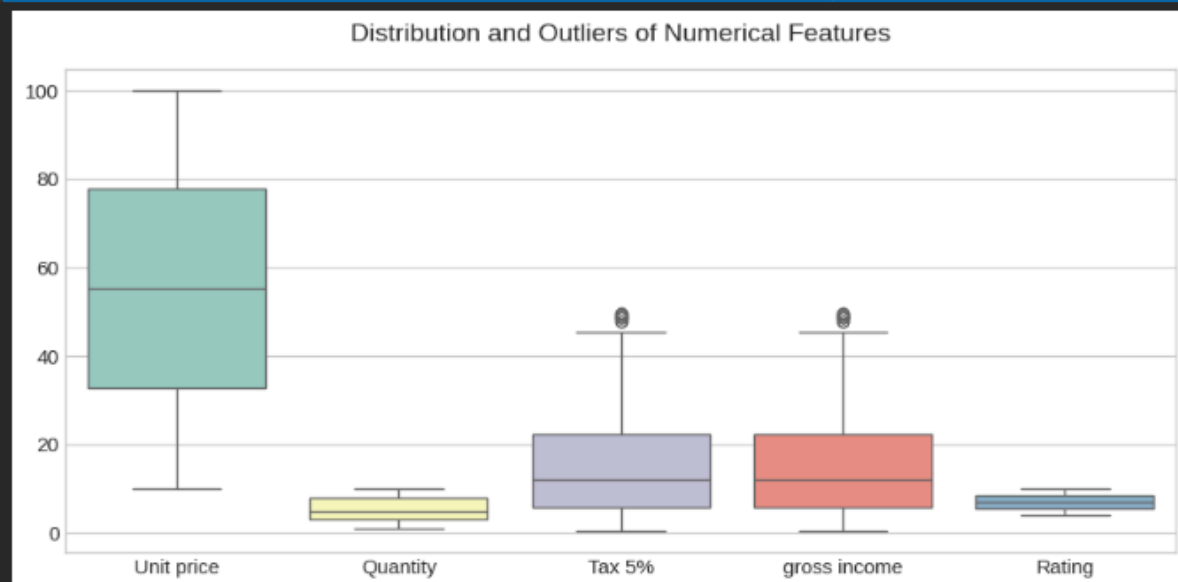
...
Categorical Feature Summaries:
Product line: 6 unique categories | Top: 'Fashion accessories' (178 transactions)
Branch: 3 unique categories | Top: 'A' (340 transactions)
City: 3 unique categories | Top: 'Yangon' (340 transactions)
Customer type: 2 unique categories | Top: 'Member' (501 transactions)
Gender: 2 unique categories | Top: 'Female' (501 transactions)
Payment: 3 unique categories | Top: 'Ewallet' (345 transactions)

```

```

# 3. Outlier Analysis Visualization
plt.figure(figsize=(9, 4.5))
sns.boxplot(data=df[['Unit price', 'Quantity', 'Tax 5%', 'gross income', 'Rating']], palette='Set3')
plt.title("Distribution and Outliers of Numerical Features")
plt.tight_layout()
plt.savefig('graphs/feature_outliers.png', dpi=200, bbox_inches='tight')
plt.show()

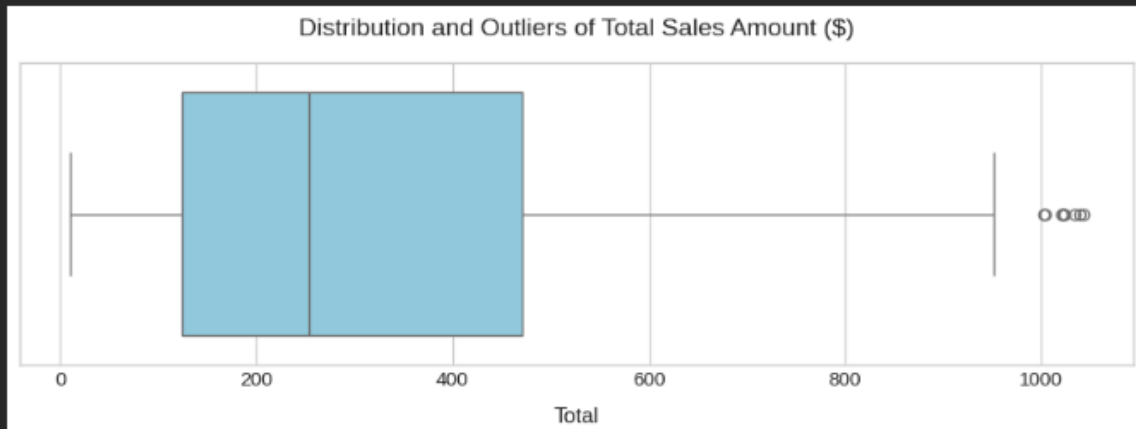
```



```

# Separate Boxplot for Total Sales Outliers
plt.figure(figsize=(9, 3.5))
sns.boxplot(data=df, x='Total', color='skyblue')
plt.title("Distribution and Outliers of Total Sales Amount ($)")
plt.tight_layout()
plt.savefig('graphs/total_sales_outliers.png', dpi=200, bbox_inches='tight')
plt.show()

```



PART 4 – CORRELATION ANALYSIS

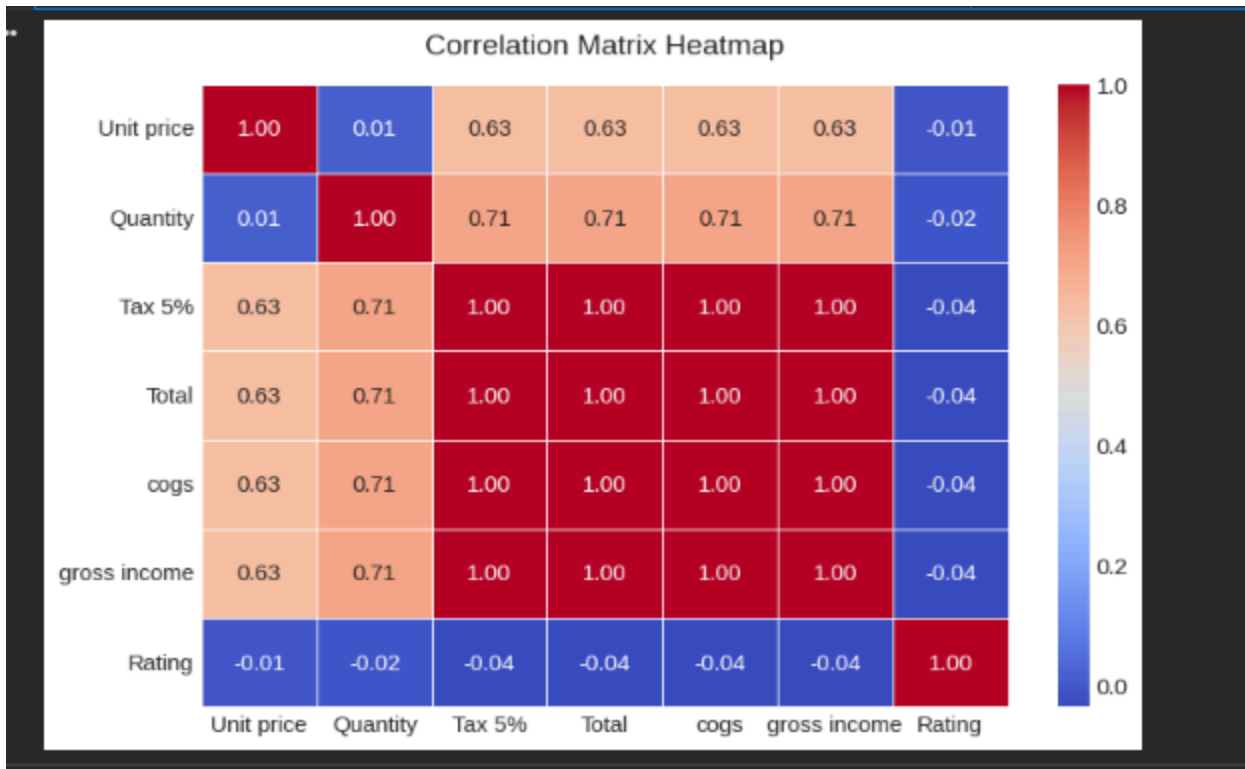
```

# Select suitable numerical features
numerical_cols = ['Unit price', 'Quantity', 'Tax 5%', 'Total', 'cogs', 'gross income', 'Rating']
corr_matrix = df[numerical_cols].corr()

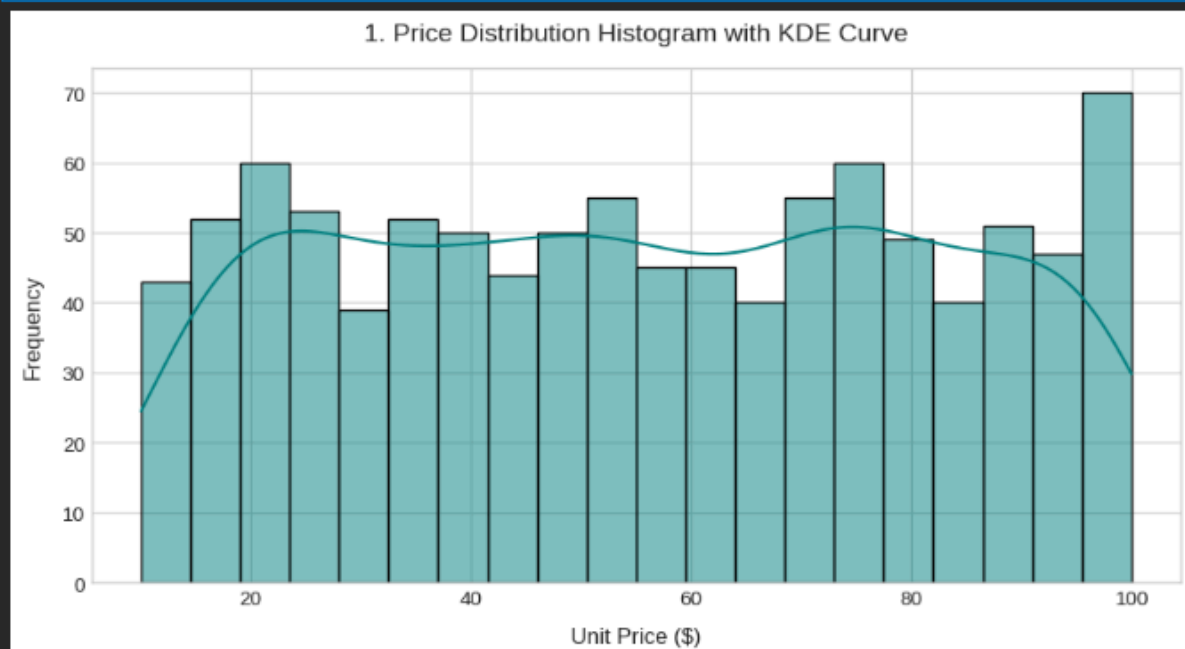
# Ensure target folder exists
os.makedirs('graphs', exist_ok=True)

# Generate Correlation Heatmap
plt.figure(figsize=(8, 5))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Matrix Heatmap")
plt.tight_layout()
plt.savefig('graphs/heatmap_correlation.png', dpi=200, bbox_inches='tight')
plt.show()

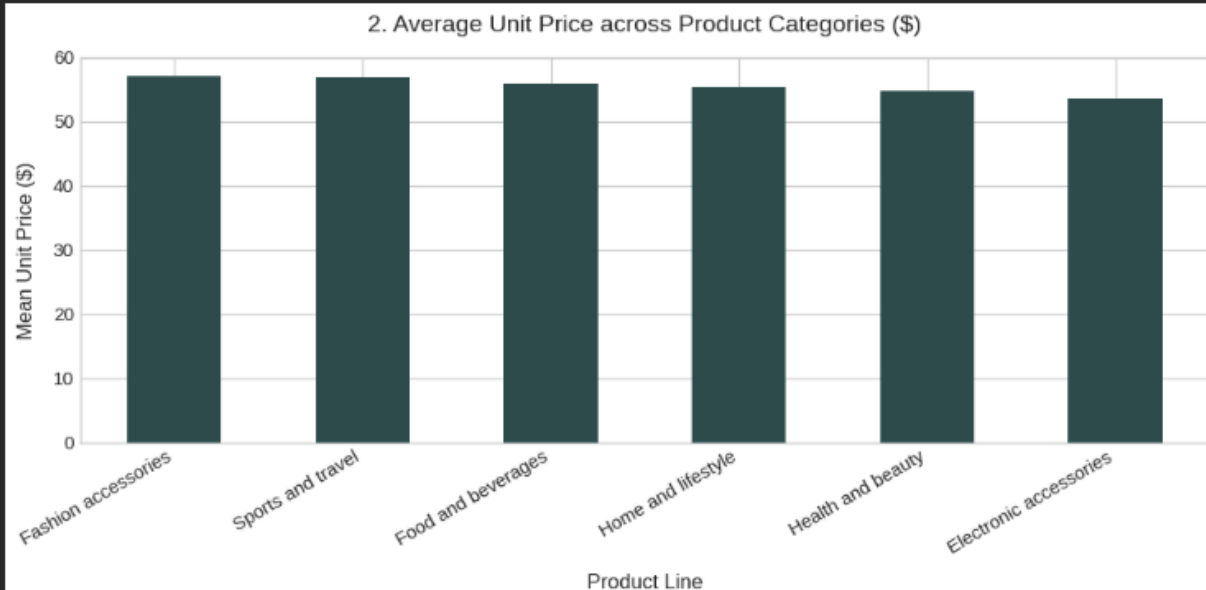
```



```
# 1. Price Distribution Histogram (KDE Curve)
plt.figure(figsize=(9, 5))
sns.histplot(df['Unit price'], kde=True, color='teal', bins=20)
plt.title('1. Price Distribution Histogram with KDE Curve')
plt.xlabel('Unit Price ($)')
plt.ylabel('Frequency')
plt.savefig('graphs/01_price_histogram.png', dpi=200, bbox_inches='tight')
plt.show()
```



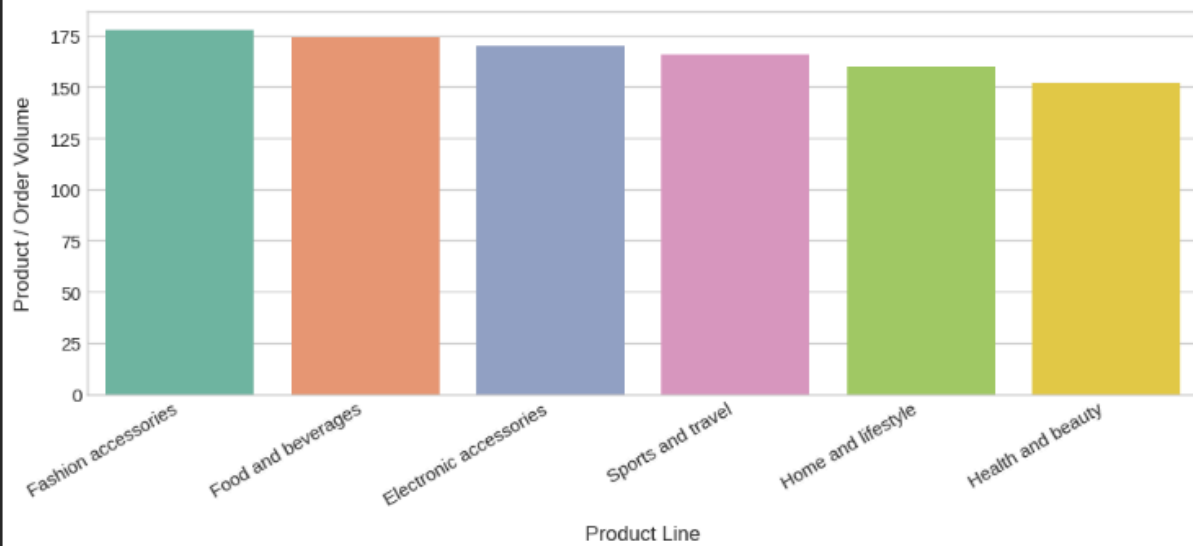
```
# 2. Average Price Bar Chart
plt.figure(figsize=(10, 5))
df.groupby('Product line')['Unit price'].mean().sort_values(ascending=False).plot(kind='bar', color='darkslategrey')
plt.title('2. Average Unit Price across Product Categories ($)')
plt.xlabel('Product Line')
plt.ylabel('Mean Unit Price ($)')
plt.xticks(rotation=30, ha='right')
plt.savefig('graphs/02_avg_price_barchart.png', dpi=200, bbox_inches='tight')
plt.show()
```



```
# 3. Category Count Plot
plt.figure(figsize=(10, 5))
sns.countplot(data=df, x='Product line', palette='Set2', order=df['Product line'].value_counts().index)
plt.title('3. Category Inventory Volume (Transaction Count Plot)')
plt.xlabel('Product Line')
plt.ylabel('Product / Order Volume')
plt.xticks(rotation=30, ha='right')
plt.savefig('graphs/03_category_countplot.png', dpi=200, bbox_inches='tight')
plt.show()
```

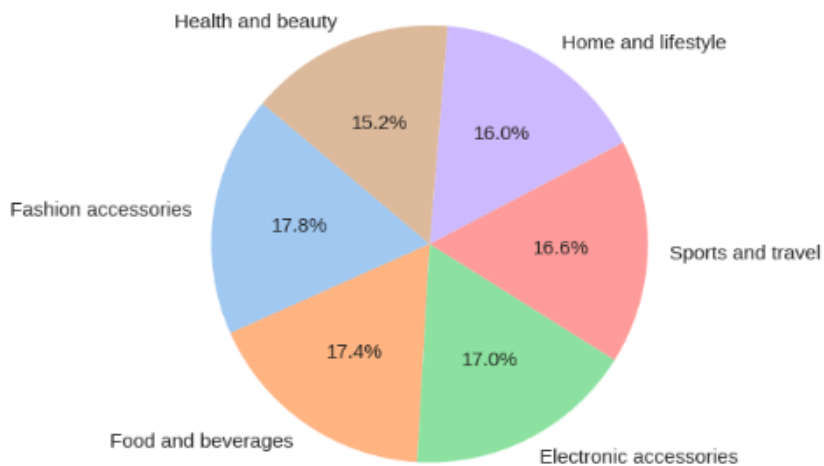
```
sns.countplot(data=df, x='Product line', palette='Set2', order=df['Product line'].value_counts().index)
```

3. Category Inventory Volume (Transaction Count Plot)



```
# 4. Category Share Pie Chart
plt.figure(figsize=(6.5, 6.5))
cat_counts = df['Product line'].value_counts()
plt.pie(cat_counts, labels=cat_counts.index, autopct='%1.1f%%', startangle=140, colors=sns.color_palette('pastel'))
plt.title('4. Percentage Distribution Share Across Product Categories')
plt.savefig('graphs/04_category_piechart.png', dpi=200, bbox_inches='tight')
plt.show()
```

4. Percentage Distribution Share Across Product Categories

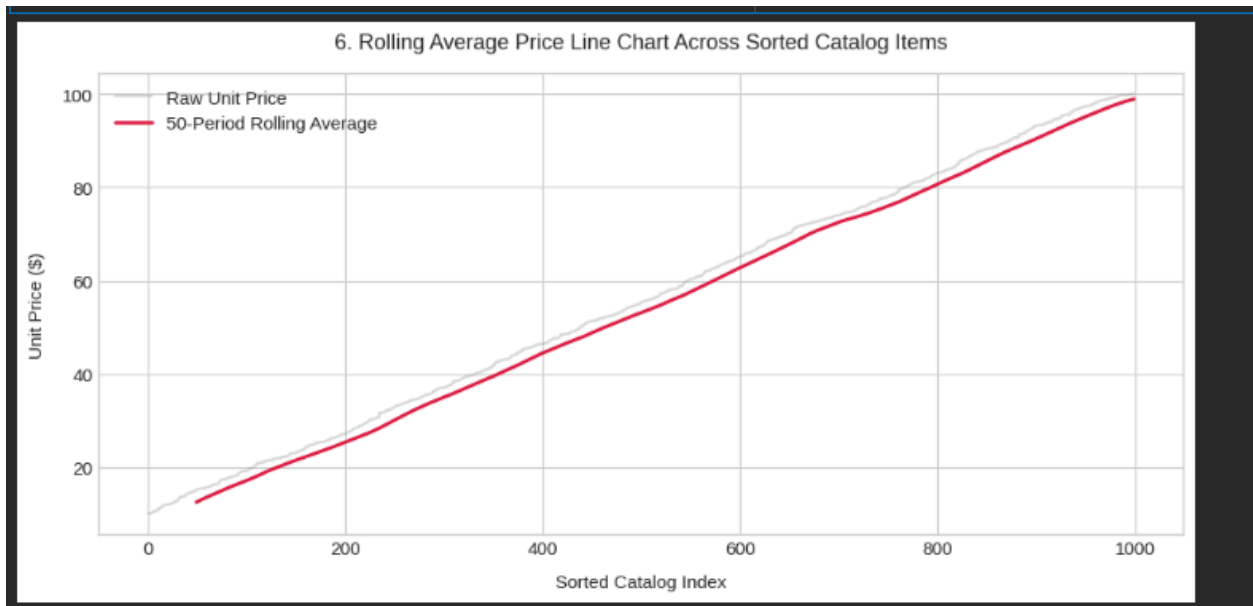


```
# 5. Price vs Rating Count (Quantity Engagement) Scatter Plot
plt.figure(figsize=(9, 5))
sns.scatterplot(data=df, x='Unit price', y='Quantity', hue='Customer type', alpha=0.7, s=60)
plt.title('5. Price vs. Customer Engagement (Quantity) Scatter Plot')
plt.xlabel('Unit Price ($)')
plt.ylabel('Quantity Purchased')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.savefig('graphs/05_price_vs_engagement_scatter.png', dpi=200, bbox_inches='tight')
plt.show()
```

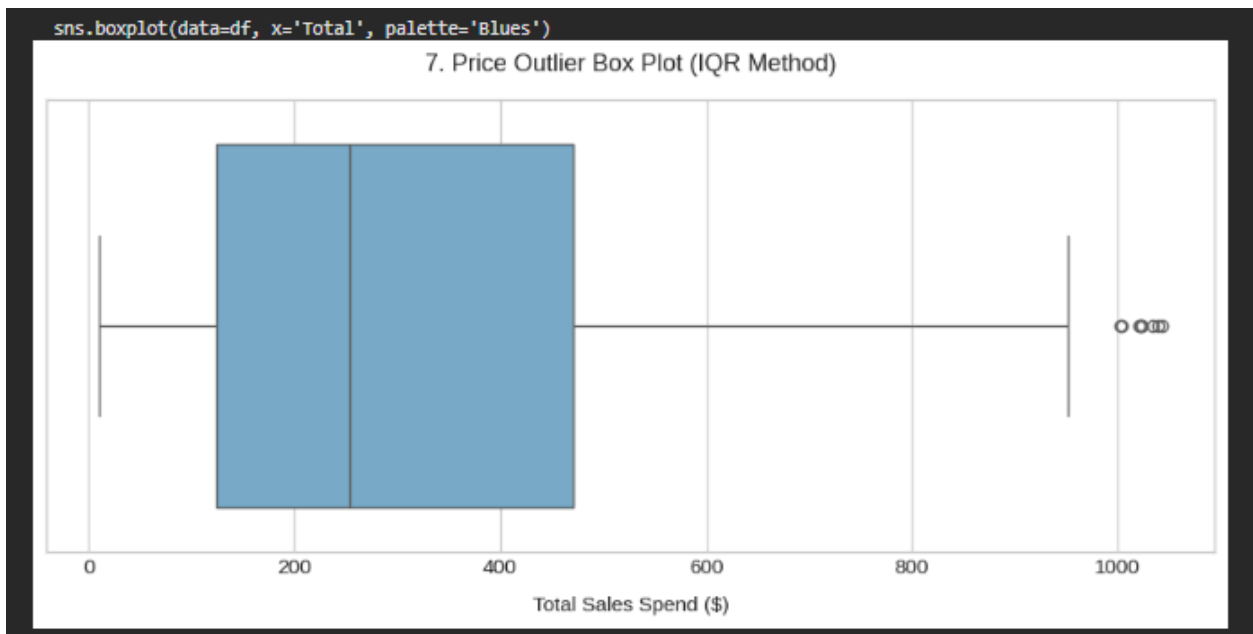


```
# 6. Rolling Average Price Line Chart
df_sorted = df.sort_values(by='Unit price').reset_index(drop=True)

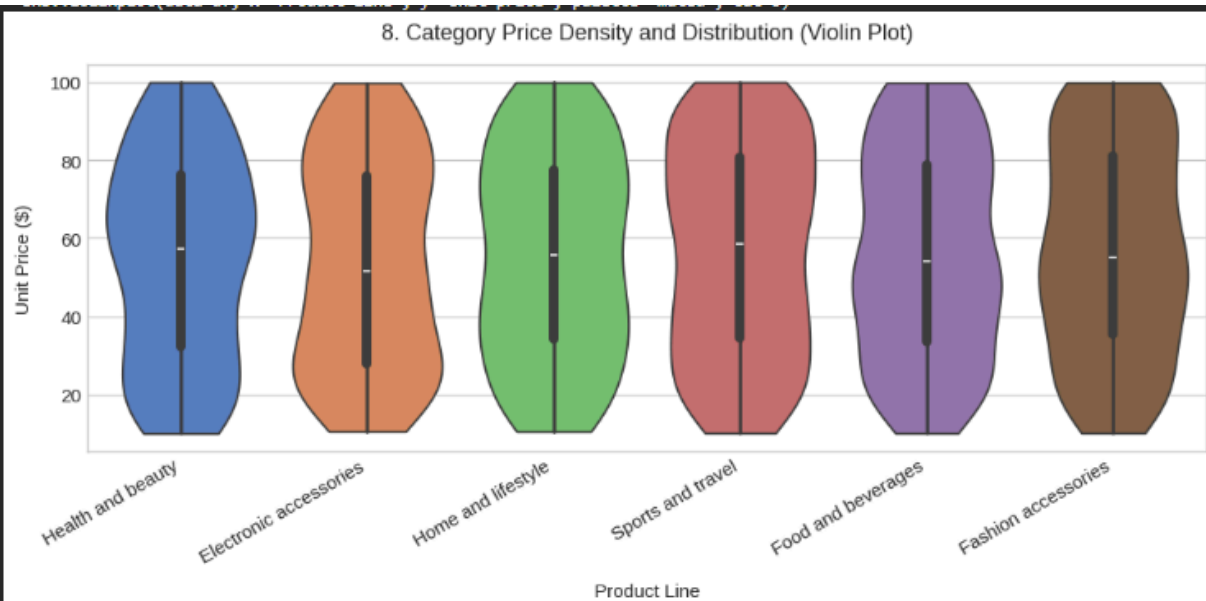
plt.figure(figsize=(10, 5))
df_sorted['Rolling_Avg_Price'] = df_sorted['Unit price'].rolling(window=50).mean()
plt.plot(df_sorted['Unit price'], alpha=0.3, color='gray', label='Raw Unit Price')
plt.plot(df_sorted['Rolling_Avg_Price'], color='crimson', linewidth=2.0, label='50-Period Rolling Average')
plt.title('6. Rolling Average Price Line Chart Across Sorted Catalog Items')
plt.xlabel('Sorted Catalog Index')
plt.ylabel('Unit Price ($)')
plt.legend(loc='upper left')
plt.savefig('graphs/06_rolling_price_linechart.png', dpi=200, bbox_inches='tight')
plt.show()
```



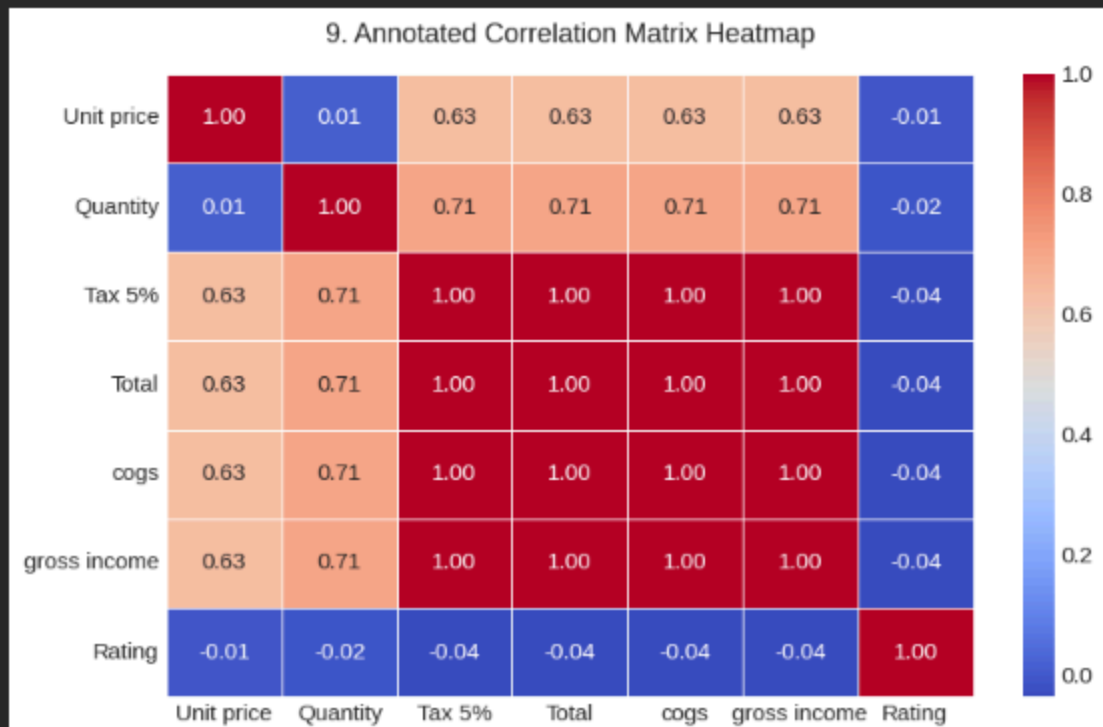
```
# 7. Price Outlier Box Plot
plt.figure(figsize=(9, 4.5))
sns.boxplot(data=df, x='Total', palette='Blues')
plt.title('7. Price Outlier Box Plot (IQR Method)')
plt.xlabel('Total Sales Spend ($)')
plt.savefig('graphs/07_price_outlier_boxplot.png', dpi=200, bbox_inches='tight')
plt.show()
```



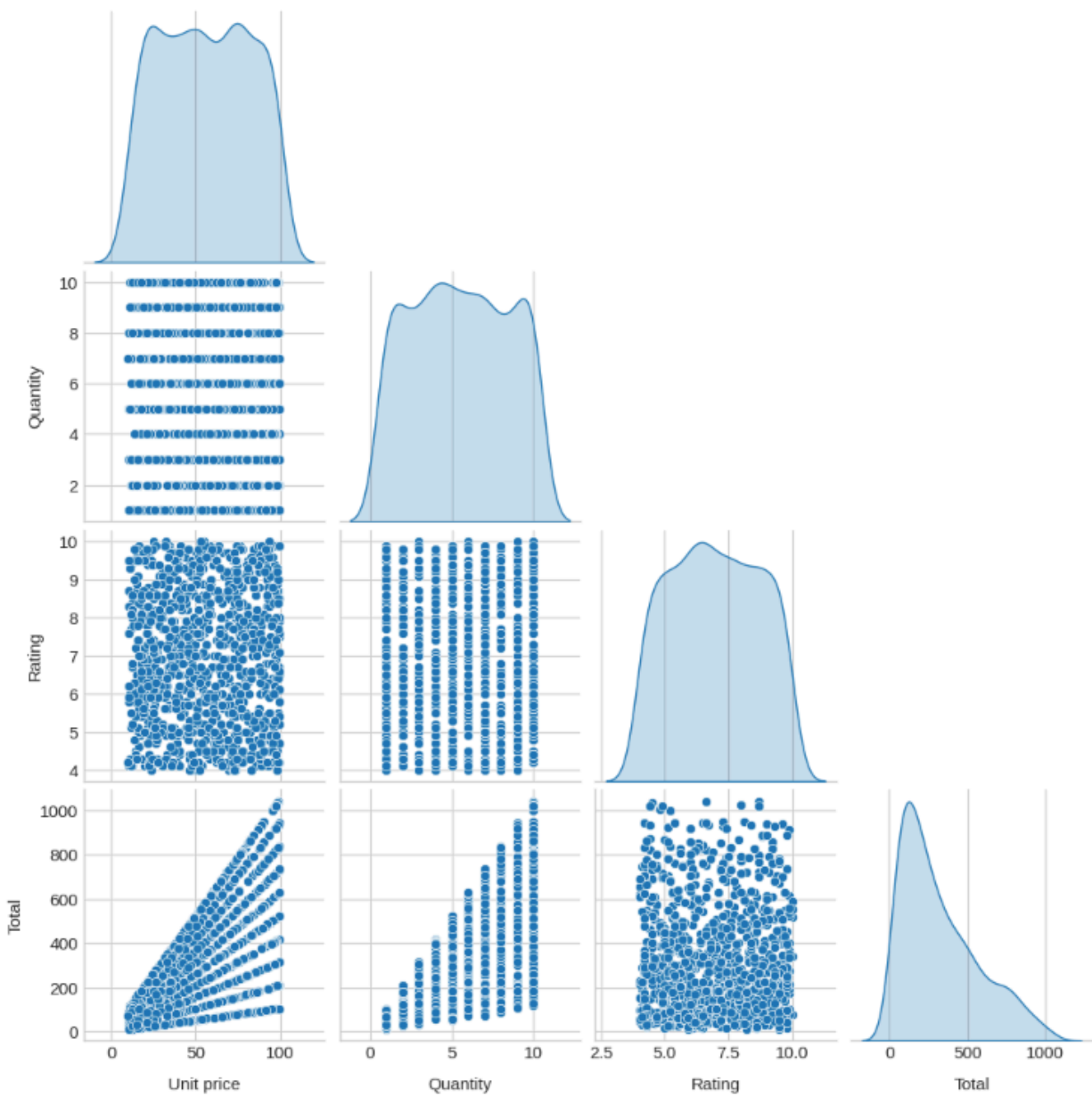
```
# 8. Category Price Violin Plot
plt.figure(figsize=(10, 5))
sns.violinplot(data=df, x='Product line', y='Unit price', palette='muted', cut=0)
plt.title('8. Category Price Density and Distribution (Violin Plot)')
plt.xlabel('Product Line')
plt.ylabel('Unit Price ($)')
plt.xticks(rotation=30, ha='right')
plt.savefig('graphs/08_category_price_violinplot.png', dpi=200, bbox_inches='tight')
plt.show()
```



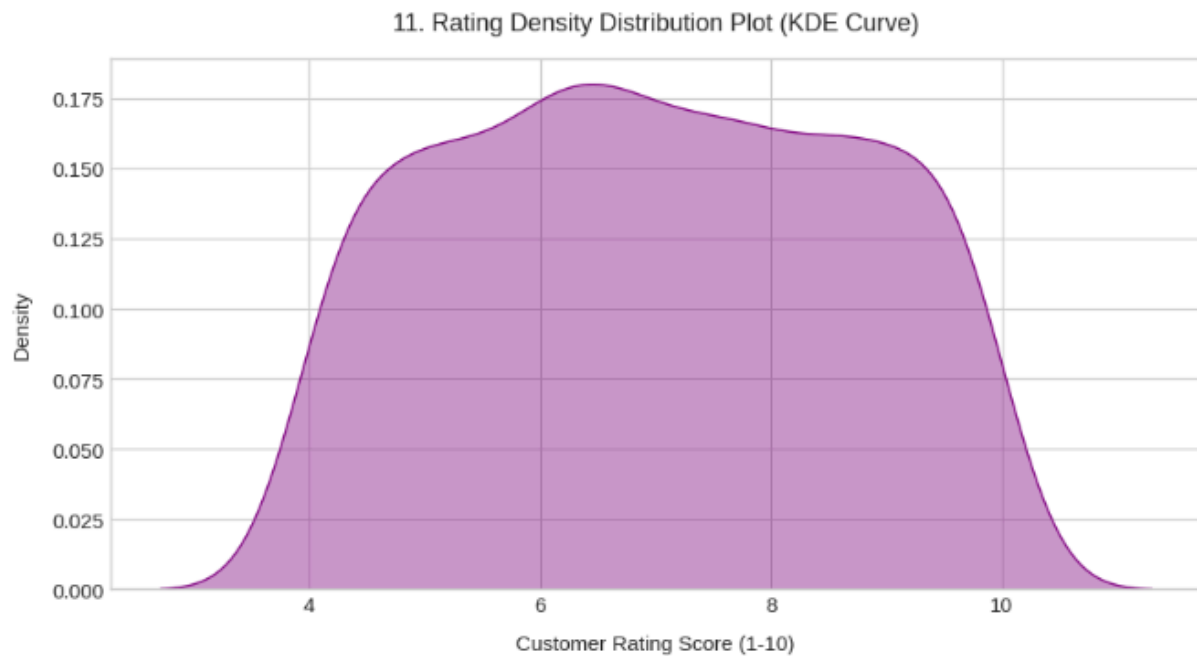
```
# 9. Correlation Matrix Heatmap
plt.figure(figsize=(8, 5))
num_cols = ['Unit price', 'Quantity', 'Tax 5%', 'Total', 'cogs', 'gross income', 'Rating']
sns.heatmap(df[num_cols].corr(), annot=True, fmt=".2f", cmap='coolwarm', linewidths=0.5)
plt.title('9. Annotated Correlation Matrix Heatmap')
plt.savefig('graphs/09_correlation_heatmap.png', dpi=200, bbox_inches='tight')
plt.show()
```



```
# 10. Multi-Variable Pair Plot
pair_cols = ['Unit price', 'Quantity', 'Rating', 'Total']
pair_grid = sns.pairplot(df[pair_cols], corner=True, diag_kind='kde')
pair_grid.fig.suptitle('10. Multi-Variable Pair Plot (Scatter & KDE Curves)', y=1.05, fontsize=14)
pair_grid.savefig('graphs/10_multivariable_pairplot.png', dpi=200, bbox_inches='tight')
plt.show()
```



```
# 11. Rating Density Distribution Plot (KDE)
plt.figure(figsize=(9, 5))
sns.kdeplot(data=df, x='Rating', fill=True, color='purple', alpha=0.4)
plt.title('11. Rating Density Distribution Plot (KDE Curve)')
plt.xlabel('Customer Rating Score (1-10)')
plt.ylabel('Density')
plt.savefig('graphs/11_rating_kde_distribution.png', dpi=200, bbox_inches='tight')
plt.show()
```



```
# 12. Cumulative Price Area Chart
plt.figure(figsize=(10, 5))
df_sorted['Cumulative_Price'] = df_sorted['Unit price'].cumsum()
plt.fill_between(range(len(df_sorted)), df_sorted['Cumulative_Price'], color='cornflowerblue', alpha=0.5)
plt.plot(range(len(df_sorted)), df_sorted['Cumulative_Price'], color='blue', linewidth=1.5)
plt.title('12. Cumulative Price Growth Area Chart Across Catalog')
plt.xlabel('Sorted Catalog Index')
plt.ylabel('Cumulative Price ($)')
plt.savefig('graphs/12_cumulative_price_areachart.png', dpi=200, bbox_inches='tight')
plt.show()
```

12. Cumulative Price Growth Area Chart Across Catalog

